Cache Design

Extra Credit: Two-Way Set Associative Cache with LRU

VHDL Code and Simulation

CMPE 413 / CMPE 640 — Custom VLSI Design, UMBC

Student: Asif Ali

Reg No: RE16458

aali12@umbc.edu

Code and report: https://github.com/aali12-eng/CMPE413-Cache-Chip-2Way

October 29, 2026

Index

1 Description of the submission....4
1.1 Design strategy....4
1.2 Results at a glance....5
2 Specification....5
3 Design overview....7
3.1 Hierarchical block diagram....7
3.2 Design styles and why....8
3.3 Entity / architecture pairs....9
4 Building blocks....11
4.1 Course primitives: dff, Dlatch, tx....11
4.2 Shared package (cache_pkg)....11
4.3 Selector (Figure 2)....11
4.4 Cache cell (Figure 1)....11
4.5 Decoder, multiplexers and comparator....12
4.6 Registers....13
4.7 Write-signal flip-flop and clock cycle counter....13
5 Cache....14
5.1 Cache block....14
5.2 Cache way....14
5.3 Hit/Miss....15
5.4 Way select....15
5.5 LRU....16
5.6 Two-way cache (32 bytes)....17
6 State machine....21
6.1 States....22
6.2 Timing....24
6.3 Inputs, outputs and connections....24
6.4 Testbench and simulation results....25
7 Chip (top level)....26
7.1 Extended chip testbench....27

8 Course testbench (chip_test)..... 30
8.1 Sample input..... 30
8.2 Output and comparison with the reference..... 31
8.3 Waveforms..... 31
9 Notes for the TA: running the simulations..... 36
9.1 Assumptions..... 37
10 Breakup of work..... 37

1 Description of the submission

This submission contains the VHDL design and simulation of the extra credit of the CMPE 413/640 project specification: "a two-way set associative cache i.e. instead of a 16 byte cache you have a 32 byte cache implementing 2 sets. For block replacement implement a least recently used (LRU) scheme." Everything else follows the main specification: byte addressable, 4-byte blocks, a 6-bit address (2-bit tag, 2-bit block offset, 2-bit byte offset), write through with no write allocate, the same chip interface, and the same timing (read hit 2 clocks, write hit or write miss 3 clocks, read miss 19 clocks).

The cache holds 2 ways (the specification's 2 sets) of 4 blocks of 4 bytes. The block offset of the address selects one block in each way, and the tag of the address is compared with the tags of both. A read miss fills an invalid way if there is one, otherwise the least recently used way; every hit and every fill makes its way the most recently used. The design reuses the direct mapped chip (the `chip design` folder) unchanged except for the cache and two control signals of the state machine.

1.1 Design strategy

- Hierarchical, structural design, built bottom-up: the course's primitive cells (`dff`, `Dlatch`, `tx`) and a small gate library form the cache cell, registers, decoders, multiplexers and counter; these form the cache, the hit/miss logic and the state machine; and these form the chip. Every level above the primitives is a netlist of instances, so each VHDL entity maps directly onto a schematic and a layout cell for Submission II.
- The cache storage uses exactly the cache cell of the specification (Figure 1): a positive level sensitive latch for Write Data, a transmission gate for Read Data, and the selector of Figure 2 that turns Chip enable and RD/WR into Write enable and Read enable. Read Data of all cells share bit lines, because a cell's transmission gate is off unless it is read.
- The chip works on the negative edge of the clock, as specified: the CPU and the memory change their signals on the positive edge, and every register of the chip samples them half a clock later. The only positive edge flip-flop makes the cache write signal, which the specification requires to be on for exactly one clock cycle from a positive edge to the next.

- Two ways built from the same cache blocks, sharing the block and byte decoders; one Hit/Miss unit per way; a way-select block that gives the hit, the way that hit and the victim way; and one LRU bit per block offset (exact LRU for two ways). The victim way is latched at the end of the tag compare, because the valid and LRU bits change while a read miss fills the block. While a read miss fills the block, each way holds its tag, valid bit and read byte, and the cache holds its hit/miss output from the end of the tag compare to the next request, so neither shows the cells being written.
- A one-hot state machine (one flip-flop per state) with a 5-bit counter sequences the four operations; its timing was taken from the specification, the desired-waveforms document and the reference output of the course testbench.
- Verification follows the hierarchy: each major block has a self-checking testbench; the chip has an extended self-checking testbench with a CPU model, a memory model and a reference model of the cache; finally the course testbench `chip_test.vhd` is run unchanged and its output is compared, line by line, with the course's reference output.

1.2 Results at a glance

- 12 of 12 self-checking testbenches pass.
- The course testbench `chip_test.vhd` runs unchanged (the component in it already matches the chip's ports) and its output matches the reference `ncsim.log` in all 70 printed clock edges (the only differences are before the first reset edge, where the reference simulator prints X and GHDL prints U for uninitialised flip-flops).
- Read hit 2 clocks, write hit and write miss 3 clocks, read miss 19 clocks, as specified.

2 Specification

Table 1: Specification summary

Item	Value
Organisation	Two-way set associative cache (extra credit)
Data	Byte addressable (8-bit data per location)
Size	2 ways ("2 sets") × 4 blocks × 4 bytes = 32 bytes
Replacement	LRU, one bit per block offset; an invalid way is filled first
Address	6 bits: 2-bit tag, 2-bit block offset, 2-bit byte offset
Write policy	Write through with no write allocate

Timing	Read hit 2 clocks; write hit / miss 3 clocks; read miss 19 clocks
Clocking	Inputs change on the positive edge; the chip works on the negative edge
Memory	Enable for one cycle with MA = tag, block offset, 00; first byte on the 8th negative edge after enable, then one byte every 2 cycles (8th, 10th, 12th, 14th)

Table 2: CPU address (CA)

Bits	**5 – 4**	**3 – 2**	**1 – 0**
Field	Tag	Block Offset	Byte Offset
Use	Compared with the tags stored in both ways	Selects 1 block in each way	Selects 1 of 4 bytes

Table 3: Chip interface (port names as in the course testbench)

Signal	**Port**	**Dir**	**Width**	**Function**
CPU Address (CA)	cpu_add	in	6	Address from the CPU, with start
CPU Data (CD)	cpu_data	inout	8	Write data in; read data out (high impedance otherwise)
Read_Write (RD/WR)	cpu_rd_wrn	in	1	1 = read, 0 = write
Start	start	in	1	Start of a CPU request
Busy	busy	out	1	Chip is processing a request
Reset	reset	in	1	Invalidates all blocks, resets the state machine
Clock	clk	in	1	Chip works on its negative edge
Memory Address (MA)	mem_add	out	6	Block address (last two bits 00) during enable
Memory Data (MD)	mem_data	in	8	Bytes from the memory on a read miss
Enable	mem_en	out	1	Memory read request, one clock cycle
Vdd, Gnd	Vdd, Gnd	in	1	Supply connections used for constant 1 and 0

3 Design overview

3.1 Hierarchical block diagram

The block diagram below shows the hierarchical breakup of this design (the diagram in the specification is generic and is only a reference). Colours: purple = chip, orange = cache, green = control, blue = building blocks, yellow = gates, grey = course primitives.

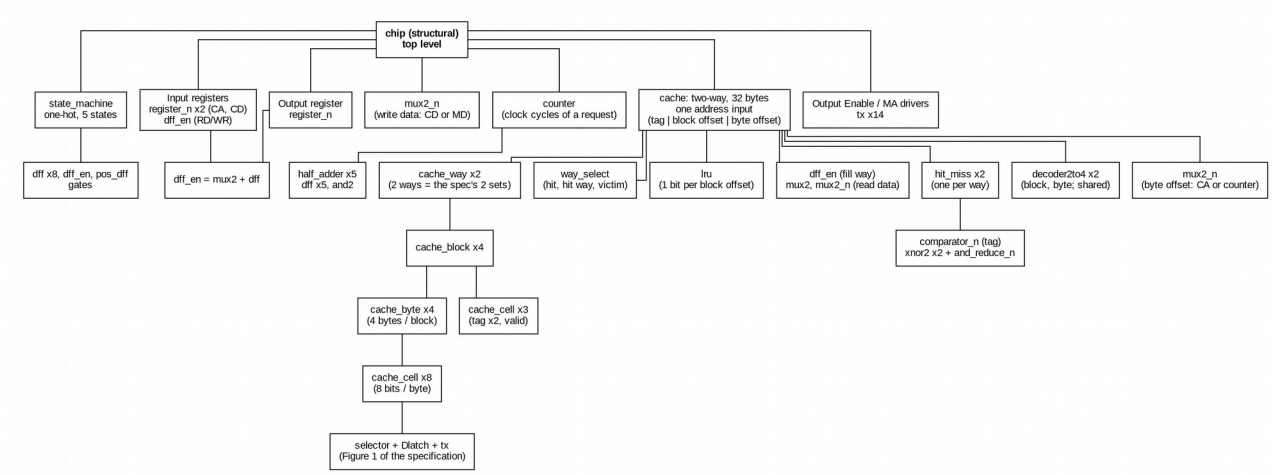


Figure 1: Hierarchical block diagram of the design

The chip connects these blocks: the CPU registers feed the state machine and the cache; the cache and its hit/miss logic tell the state machine whether the request hits; the state machine drives busy, the memory enable, the output register and the cache write signal.

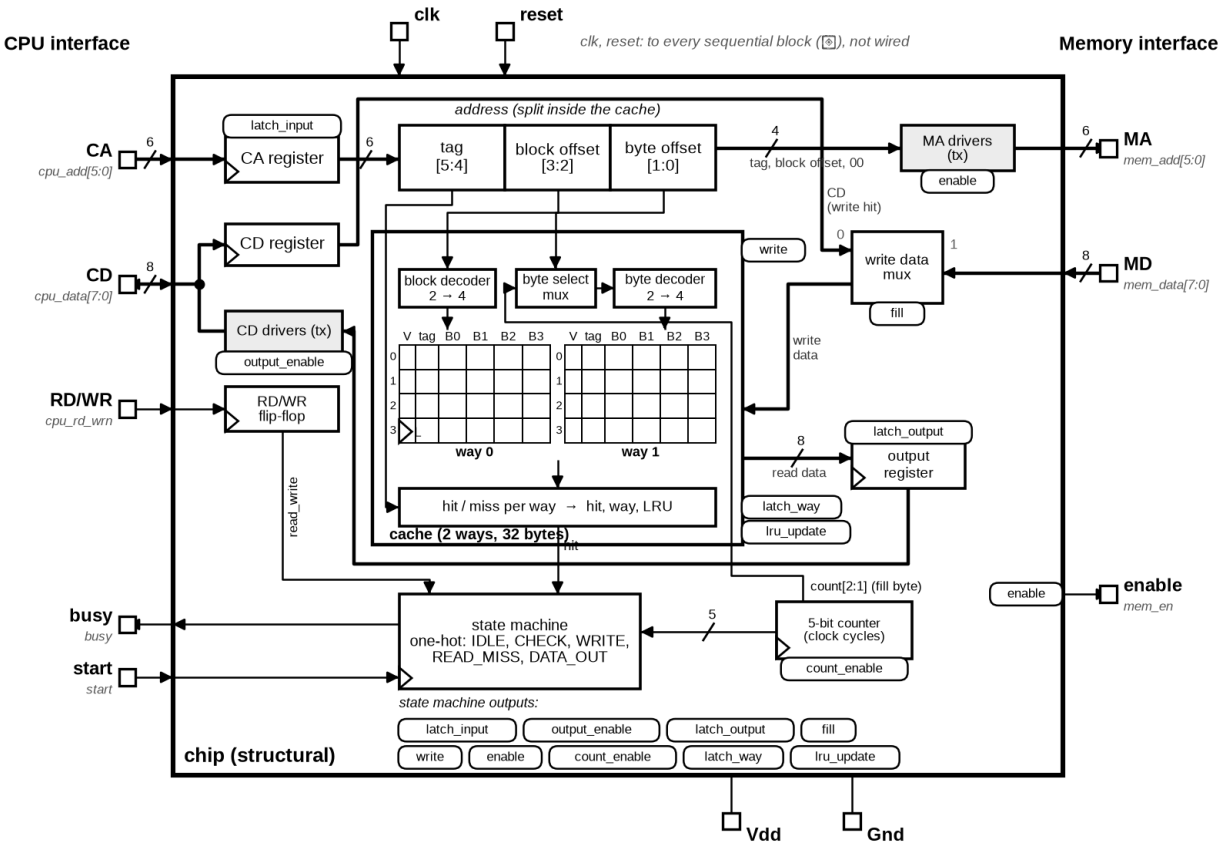


Figure 2: Chip overview: pins and data path

The following chapters describe each part of the design: its function, its testbench and the result. A few key diagrams are shown: the cache cell of the specification, the cache way and the two-way cache, and the control flow of the cache, the state machine and the chip. The VHDL of every part is in the folder code/two_way of the submission.

How to read the block diagrams in this report: grey arrow shapes are module ports; thick lines are buses, labelled with their width; a dot marks where a net branches. A triangle at the bottom-left of a block marks a sequential block, one that stores state (it contains negative edge flip-flops or, with an "L" next to the triangle, level sensitive latches: the cache cells). Blocks without the triangle are combinational. The clock is not drawn; it reaches every sequential block.

3.2 Design styles and why

- Structural VHDL (entity instantiation and generate loops) for every block above the primitives: it describes exactly the gates and cells that will be drawn in the

schematic and layout in Submission II, so the schematics can be imported and LVS compared directly.

- Behavioural primitives only where the course provides them (`dff`, `Dlatch`, `tx`) and for the basic gates (`y <= a and b after GATE_DELAY`), so every gate has a propagation delay. `GATE_DELAY` = 100 ps, a conservative figure for the 45 nm (gpdk045) process; the longest path in one clock cycle is about 12 gates (1.2 ns) against a 10 ns clock.
- Negative edge registers everywhere: the CPU and memory signals are set up half a clock before they are sampled, which avoids setup time violations (as the specification intends).
- A one-hot state machine: one flip-flop per state, so every state decode is a single flip-flop output and the next-state logic is a few AND/OR gates per state; this is simple to lay out and has no decoding glitches on the outputs.
- Tri-state buses built from transmission gates: the cache cells share bit lines; CD is driven only while output enable is on; MA is driven only while enable is on (as in the reference output).
- Glitch-free cache writes: the cache address and data come from registers (or the memory) and are stable from half a cycle before the write signal until half a cycle after it, and the write signal itself comes straight from a flip-flop.

3.3 Entity / architecture pairs

The code and simulation results in Sections 4 to 7 are given in the same order as this list.

Table 4: Entity / architecture pairs

#	Entity	Architecture	File	Block	Testbench
1	dff	structural	two_way/ primitives/ dff.vhd	Course primitive: negative edge D flip-flop	— (primitive)
2	Dlatch	structural	two_way/ primitives/ Dlatch.vhd	Course primitive: positive level D latch	— (primitive)
3	tx	structural	two_way/ primitives/tx.vhd	Course primitive: transmission gate	— (primitive)
4	not_gate, and2_gate, or2_gate, nand2_gate, nor2_gate, xor2_gate, xnor2_gate	rtl	two_way/gates/ *.vhd	Basic gates (100 ps delay)	— (primitive)
5	and3_gate, or3_gate, or4_gate	structural	two_way/gates/ *.vhd	Basic gates (from 2-input gates)	— (primitive)
6	selector	structural	two_way/blocks/	Cache cell:	tb_selector

			selector.vhd	selector (Figure 2)	
7	cache_cell	structural	two_way/blocks/ cache_cell.vhd	Cache cell (Figure 1)	tb_cache_cell
8	mux2, mux2_n	structural	two_way/blocks/ mux2*.vhd	Mux (byte offset, write data, way, read data, register load)	— (primitive)
9	mux4_n	structural	two_way/blocks/ mux4_n.vhd	4:1 mux (LRU bit of the block offset)	— (primitive)
10	decoder2to4	structural	two_way/blocks/ decoder2to4.vhd	Decoder (block offset, byte offset)	— (primitive)
11	and_reduce_n, comparator_n	structural	two_way/blocks/ *.vhd	Hit/Miss: tag comparator	— (primitive)
12	dff_en	structural	two_way/blocks/ dff_en.vhd	Register bit with load	— (primitive)
13	register_n	structural	two_way/blocks/ register_n.vhd	Registers (CA, CD, output register)	tb_register_n
14	pos_dff	structural	two_way/blocks/ pos_dff.vhd	State machine: write signal flip-flop	— (primitive)
15	half_adder	structural	two_way/blocks/ half_adder.vhd	Counter bit	— (primitive)
16	counter	structural	two_way/blocks/ counter.vhd	Clock cycle counter of every request	tb_counter
17	cache_byte	structural	two_way/cache/ cache_byte.vhd	Cache: one byte (8 cache cells)	— (primitive)
18	cache_block	structural	two_way/cache/ cache_block.vhd	Cache block (valid, tag, 4 bytes)	tb_cache_block
19	cache_way	structural	two_way/cache/ cache_way.vhd	Cache way (4 blocks; 2 per cache)	tb_cache_way
20	hit_miss	structural	two_way/cache/ hit_miss.vhd	Cache: Hit/Miss of one way	tb_hit_miss
21	way_select	structural	two_way/cache/ way_select.vhd	Cache: hit, hit way, victim	tb_way_select
22	lru	structural	two_way/cache/ lru.vhd	Cache: LRU bits	tb_lru
23	cache	structural	two_way/cache/ cache.vhd	Two-way cache (address split, decoders, 2 ways, Hit/Miss, LRU)	tb_cache
24	state_machine	structural	two_way/control/ state_machine.vhd	State Machine	tb_state_machine
25	chip	structural	two_way/ chip.vhd	Top level (the chip)	tb_chip, chip_test

4 Building blocks

4.1 Course primitives: dff, Dlatch, tx

These three cells are the course's `useful_vhdl_files`, used unchanged: a negative edge triggered D flip-flop, a positive level sensitive D latch and a transmission gate (output high impedance unless `sel` = 1 and `selnot` = 0). Testbenches for low level primitive cells are not required.

4.2 Shared package (cache_pkg)

cache_pkg holds the design's constants (address, data, tag and counter widths) and GATE_DELAY, the 100 ps propagation delay of the basic gates. The basic gates (not, and2, or2, nand2, nor2, xor2, xnor2, and3, or3, or4) are one assignment each, or built from 2-input gates; as the lowest level of the design, their code, circuits and waveforms are not repeated here.

4.3 Selector (Figure 2)

The selector of the cache cell implements the truth table of Figure 2 of the specification: Read enable = Chip enable · RD/WR, Write enable = Chip enable · not RD/WR.

Testbench

- All rows of the truth table of Figure 2 (Chip enable 0 with RD/WR 0 and 1; read; write).

Simulation results

Simulation result: **PASS** — `tb_selector`: all checks passed, simulation ended at 40 ns.

4.4 Cache cell (Figure 1)

One bit of the cache, exactly as Figure 1 of the specification: Write Data goes into a positive level sensitive latch whose clock is the selector's Write enable; the latch output goes through a transmission gate, opened by Read enable, to Read Data. When the cell is not read, Read Data is high impedance, so many cells can drive one bit line.

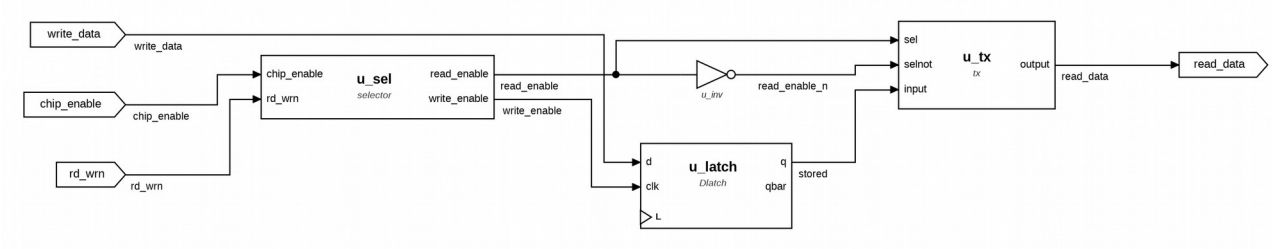


Figure 3: `cache_cell`: internal structure

Testbench

- Write 1, read it back; write 0, read it back.
- With Chip enable = 0: Read Data is high impedance and a write request does not change the cell.
- As in the chip, RD/WR and Write Data change only while Chip enable is off, so the selector never produces a write glitch.

Simulation results

Simulation result: **PASS** — `tb_cache_cell`: all checks passed, simulation ended at 90 ns.

4.5 Decoder, multiplexers and comparator

Low-level combinational blocks built from the basic gates: the 2-to-4 decoder (block offset and byte offset), the 1-bit and N-bit 2:1 multiplexers (cache write address and data, register load), and the N-bit equality comparator used for the tag compare.

Simulation results

Simulation result: **PASS** — `tb_decoder2to4`: all checks passed, simulation ended at 80 ns.

Simulation result: **PASS** — `tb_mux2`: all checks passed, simulation ended at 80 ns.

Simulation result: **PASS** — `tb_mux2_n`: all checks passed, simulation ended at 100 ns.

Simulation result: **PASS** — `tb_mux4_n`: all checks passed, simulation ended at 120 ns.

Simulation result: **PASS** — `tb_and_reduce_n`: all checks passed, simulation ended at 160 ns.

Simulation result: **PASS** — `tb_comparator_n`: all checks passed, simulation ended at 10240 ns.

4.6 Registers

`dff_en` is a register bit with load: a `mux2` feeds back the flip-flop output when load = 0. `register_n` is N of them; the chip uses one for the CPU address (6 bits), one for the CPU data (8 bits) and one for the output register (8 bits), plus a `dff_en` for RD/WR.

Testbench

- N = 8: load, hold twice with new data at the input, load again; inputs change on the positive edge and q is checked after the negative edge.

Simulation results

Simulation result: **PASS** — `tb_register_n`: all checks passed, simulation ended at 60 ns.

Simulation result: **PASS** — `tb_dff_en`: all checks passed, simulation ended at 70 ns.

4.7 Write-signal flip-flop and clock cycle counter

`pos_dff is the course dff clocked by the inverted clock, i.e. a positive edge flip-flop; it makes the cache write signal. counter is a 5-bit synchronous counter (half adders and negative edge flip-flops): restart = 1 loads 1, count = 1 counts up, and count = 0 with restart = 0 clears it. The state machine counts the clock cycles of every request with it: the counter starts at 1 on the negative edge that accepts a request and, after the edge where busy goes low, holds 2 (read hit), 3 (write hit or miss) or 19 (read miss).`

Testbench

- Clear, count from 1 to 20 (past 19, a read miss) and clear; restart loads 1 and the count runs to 19; restart straight from 19 (a back-to-back request) loads 1 again; then clear.

Simulation results

Simulation result: **PASS** — `tb_counter`: all checks passed, simulation ended at 430 ns.

Simulation result: **PASS** — `tb_pos_dff`: all checks passed, simulation ended at 35 ns.

Simulation result: **PASS** — `tb_half_adder`: all checks passed, simulation ended at 40 ns.

5 Cache

5.1 Cache block

A cache block is 4 bytes (`cache_byte`, 8 cache cells each), a 2-bit tag and a valid bit, all cache cells. Byte i is enabled by `block_select` · `byte_select(i)`; the tag and valid cells by `block_select`. Reset also enables the valid cell, so a single reset pulse clears all four valid bits. The cell outputs drive shared buses (`read_data`, `tag_out`, `valid_out`) that are high impedance when the block is not selected.

Testbench

- Reset clears valid.
- Write the 4 bytes with the tag and valid (as a read miss does), read every byte back.
- A single-byte write (write hit) changes only that byte and not the tag.
- Block not selected: all its buses are high impedance; reset clears valid again.

Simulation results

Simulation result: PASS — `tb_cache_block`: all checks passed, simulation ended at 165 ns.

Simulation result: PASS — `tb_cache_byte`: all checks passed, simulation ended at 210 ns.

5.2 Cache way

One way of the cache: 4 cache blocks (16 bytes). The block select and byte select come from the cache's decoders, which both ways share. The way's own write signal writes the selected byte; during a read miss (fill = 1) it also writes the tag and sets the valid bit. The selected byte, tag and valid bit are always read onto the way's buses. During a read miss (fill = 1) a Dlatch on every output (8 data bits, 2 tag bits, the valid bit) holds the values from before the fill: while a tag, valid or data cell is written its Read Data is high impedance, and the byte select steps through the bytes being written, so without the latches the hit/miss logic would see Z and X until the block is complete. The latches pass the cells again as soon as fill = 0, for the read of the requested byte.

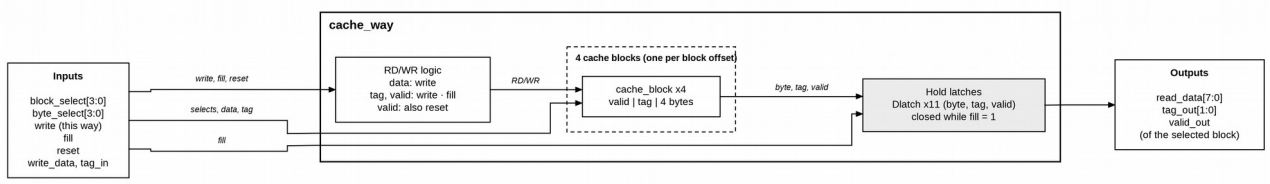


Figure 4: `cache_way`: block diagram

Testbench

- Reset invalidates every block.
- A read miss into block 1 with tag 10 (4 bytes, tag and valid), read back.
- Other blocks untouched; a write hit to one byte keeps the tag and valid bit; reset again.

Simulation results

Simulation result: PASS — `tb_cache_way`: all checks passed, simulation ended at 180 ns.

5.3 Hit/Miss

Hit/Miss of one way: hit = valid · (tag of the address = tag stored in the block of that way); miss = not hit. The tag compare is a 2-bit `comparator_n`. The cache has one per way.

Testbench

- All 32 combinations of address tag, stored tag and valid.

Simulation results

Simulation result: PASS — `tb_hit_miss`: all checks passed, simulation ended at 320 ns.

5.4 Way select

Combines the two ways: hit = hit0 + hit1, miss = not hit, hit_way = hit1 (the way that holds the block). The victim (the way a read miss fills) is an invalid way first, otherwise the LRU way: victim = valid0 · (not valid1 + lru_way).

Testbench

- All 32 combinations of hit0, hit1, valid0, valid1 and lru_way against a reference model.

Simulation results

Simulation result: PASS — `tb_way_select`: all checks passed, simulation ended at 320 ns.

5.5 LRU

One LRU bit per block offset, held in negative edge registers: the bit is the way to replace next. When update = 1, the bit of the addressed block offset becomes the other way (lru <= not used_way); a decoder enabled by update loads only that bit. A 4:1 mux reads the bit of the addressed block offset. Reset clears all bits. With two ways one bit gives exact LRU.

Testbench

- Reset; a block used in way 0 then way 1; update = 0 holds; every block offset updated; reset again — every bit read back after each step.

Simulation results

Simulation result: PASS — `tb_lru`: all checks passed, simulation ended at 100 ns.

Truth tables

Every gate output of `lru` for all 32 input combinations. bits(i)+ is the value of bit i after the next negative edge of clk; – means the bit holds its value (load(i) = 0), and bi is the present value of bit i.

Table 5: `lru`: every gate output for every input (bits(3..0)+ after the next negative edge)

reset	update	block offset	used way	set_en(3..0) u_dec	used_nu_inv	rst_nu_rst_n	d u_d	load(3..0) u_load	bits(3..0)+ u_bit	lru_way u_mux
0	0	00	0	0000	1	1	1	0000	– – – –	b0
0	0	00	1	0000	0	1	0	0000	– – – –	b0
0	0	01	0	0000	1	1	1	0000	– – – –	b1
0	0	01	1	0000	0	1	0	0000	– – – –	b1
0	0	10	0	0000	1	1	1	0000	– – – –	b2
0	0	10	1	0000	0	1	0	0000	– – – –	b2
0	0	11	0	0000	1	1	1	0000	– – – –	b3
0	0	11	1	0000	0	1	0	0000	– – – –	b3
0	1	00	0	0001	1	1	1	0001	– – – 1	b0
0	1	00	1	0001	0	1	0	0001	– – – 0	b0
0	1	01	0	0010	1	1	1	0010	– – 1 –	b1
0	1	01	1	0010	0	1	0	0010	– – 0 –	b1
0	1	10	0	0100	1	1	1	0100	– 1 – –	b2
0	1	10	1	0100	0	1	0	0100	– 0 – –	b2
0	1	11	0	1000	1	1	1	1000	1 – – –	b3
0	1	11	1	1000	0	1	0	1000	0 – – –	b3
1	0	00	0	0000	1	0	0	1111	0 0 0 0	b0
1	0	00	1	0000	0	0	0	1111	0 0 0 0	b0
1	0	01	0	0000	1	0	0	1111	0 0 0 0	b1
1	0	01	1	0000	0	0	0	1111	0 0 0 0	b1
1	0	10	0	0000	1	0	0	1111	0 0 0 0	b2
1	0	10	1	0000	0	0	0	1111	0 0 0 0	b2
1	0	11	0	0000	1	0	0	1111	0 0 0 0	b3
1	0	11	1	0000	0	0	0	1111	0 0 0 0	b3
1	1	00	0	0001	1	0	0	1111	0 0 0 0	b0
1	1	00	1	0001	0	0	0	1111	0 0 0 0	b0
1	1	01	0	0010	1	0	0	1111	0 0 0 0	b1
1	1	01	1	0010	0	0	0	1111	0 0 0 0	b1
1	1	10	0	0100	1	0	0	1111	0 0 0 0	b2
1	1	10	1	0100	0	0	0	1111	0 0 0 0	b2

1	1	11	0	1000	1	0	0	1111	0 0 0 0	b3
1	1	11	1	1000	0	0	0	1111	0 0 0 0	b3

One LRU bit, `u_bit(i)` (`dff_en`: a mux2 that keeps q when load = 0, and a negative edge dff):

Table 6: `dff_en` in `lru`: one LRU bit

load(i)	d	bits(i)	mux2 output	bits(i)+
0	x	0	0	0
0	x	1	1	1
1	0	x	0	0
1	1	x	1	1

Read, `u_mux` (`mux4_n`, combinational): lru_way = bits(block offset).

Table 7: `mux4_n` in `lru`: LRU way of the addressed block offset

block offset	lru_way
00	bits(0)
01	bits(1)
10	bits(2)
11	bits(3)

With the state machine (CHECK cycle): lru_update = CHECK · (hit + RD/WR) and used_way = hit ? hit_way : victim, so the addressed bit becomes not used_way after a read hit, write hit or read miss, and does not change after a write miss.

Table 8: `lru` update for each kind of access

Access	hit	RD/WR	lru_update	used_way	Addressed bit after
Read hit in way 0	1	1	1	0 (hit_way)	1
Read hit in way 1	1	1	1	1 (hit_way)	0
Read miss, victim way 0	0	1	1	0 (victim)	1
Read miss, victim way 1	0	1	1	1 (victim)	0
Write hit in way 0	1	0	1	0 (hit_way)	1
Write hit in way 1	1	0	1	1 (hit_way)	0
Write miss	0	0	0	x	unchanged

5.6 Two-way cache (32 bytes)

The cache has a single address input, the latched CPU address, and splits it into tag (bits 5–4), block offset (3–2) and byte offset (1–0). The shared decoders select the block and the byte in both ways at once; during a read miss a mux2_n takes the byte

offset from fill_byte (the counter). Each way's Hit/Miss compares the tag; way_select gives hit, hit_way and the victim. The cache's hit and miss outputs come through a Dlatch that is open only while latch_way (the CHECK cycle) or reset is 1: the state machine reads the compare at the end of CHECK, and the outputs keep that result until the next request's CHECK; reset clears them.

Replacement: on the negative edge that ends the tag compare, `latch_way` keeps the victim in the `fill_way` register (the valid and LRU bits change during the fill), and `lru_update` makes the way used (the way that hit, or the victim of a read miss) the most recently used. The write signal goes to the way that hit (write hit) or to `fill_way` (read miss). The byte read comes from the way that hit.

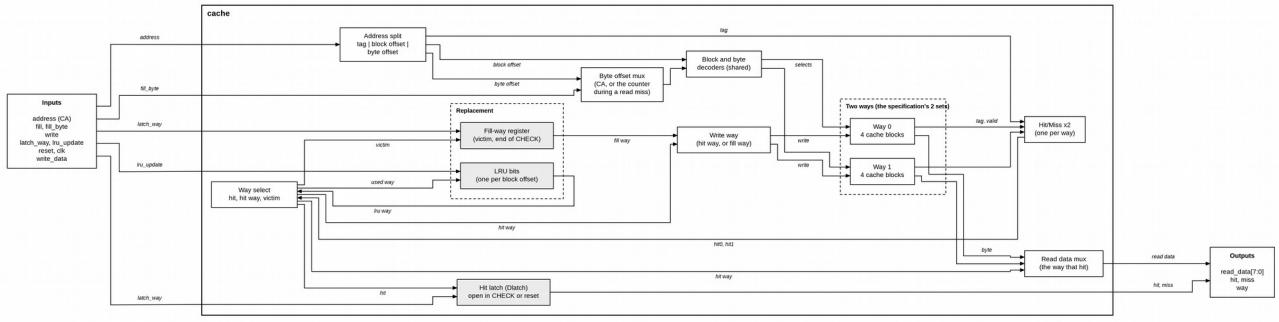


Figure 5: `cache`: block diagram

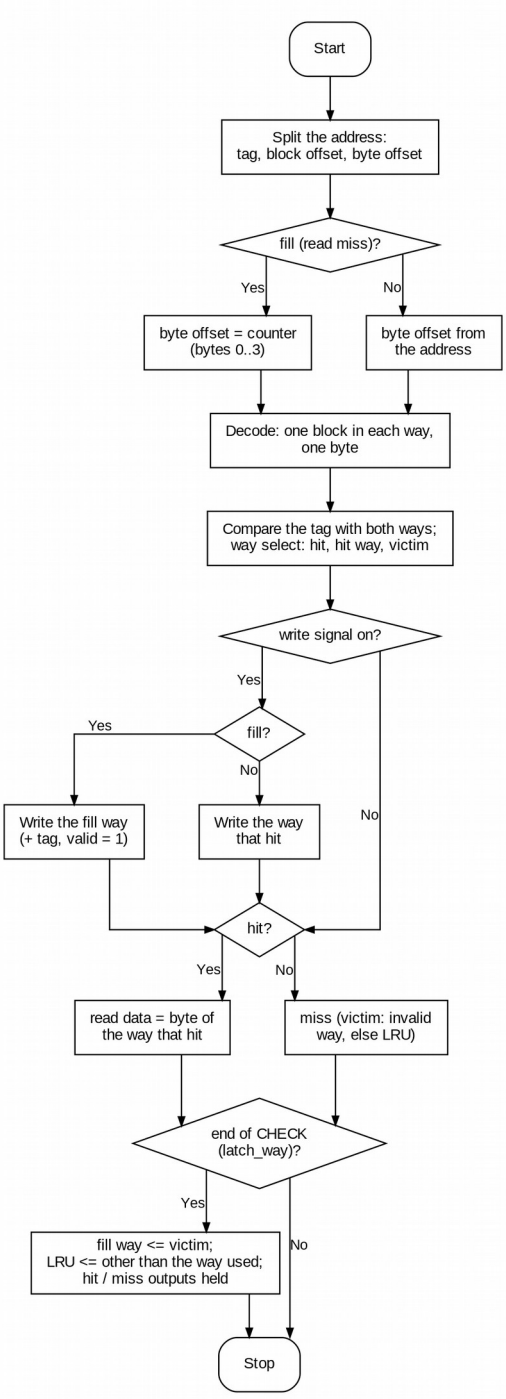


Figure 6: cache: operation flowchart

Testbench

- Reset; A (tag 0, block 1) read miss fills way 0; B (tag 1, same block) fills way 1.
- Both A and B hit, each read from its own way.
- C (tag 2) read miss replaces the LRU way (B); B then misses (a write miss, which does not change the LRU); A and C hit.

- A write hit writes only the way that hit; another block offset misses; reset empties the cache.

Simulation results

Simulation result: **PASS** — `tb_cache`: all checks passed, simulation ended at 450 ns.

6 State machine

The state machine is one-hot: five negative edge flip-flops, one per state, with the next-state logic built from gates. The counter gives the clock cycle c of the request: c = 1 after the negative edge that accepts the request (CHECK); it counts in CHECK, WRITE and READ_MISS, and after the edge where busy goes low it holds 2 (read hit), 3 (write) or 19 (read miss). It is cleared on the next negative edge, or started at 1 again by a new request.

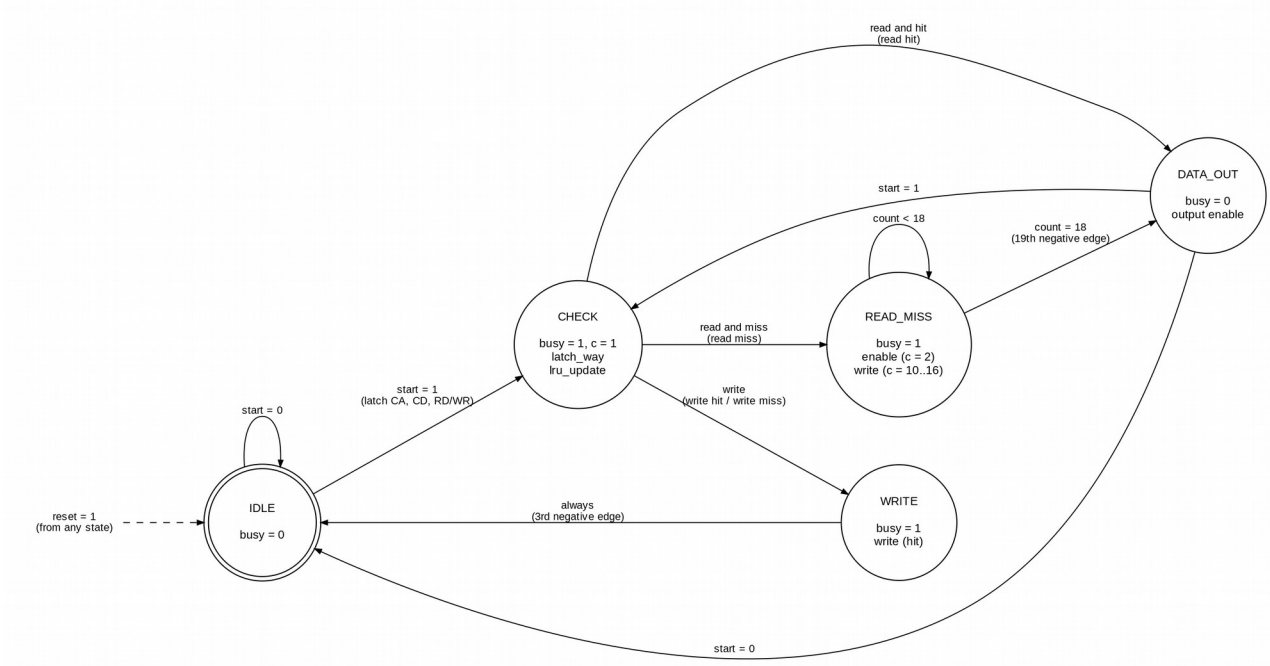


Figure 7: State machine diagram

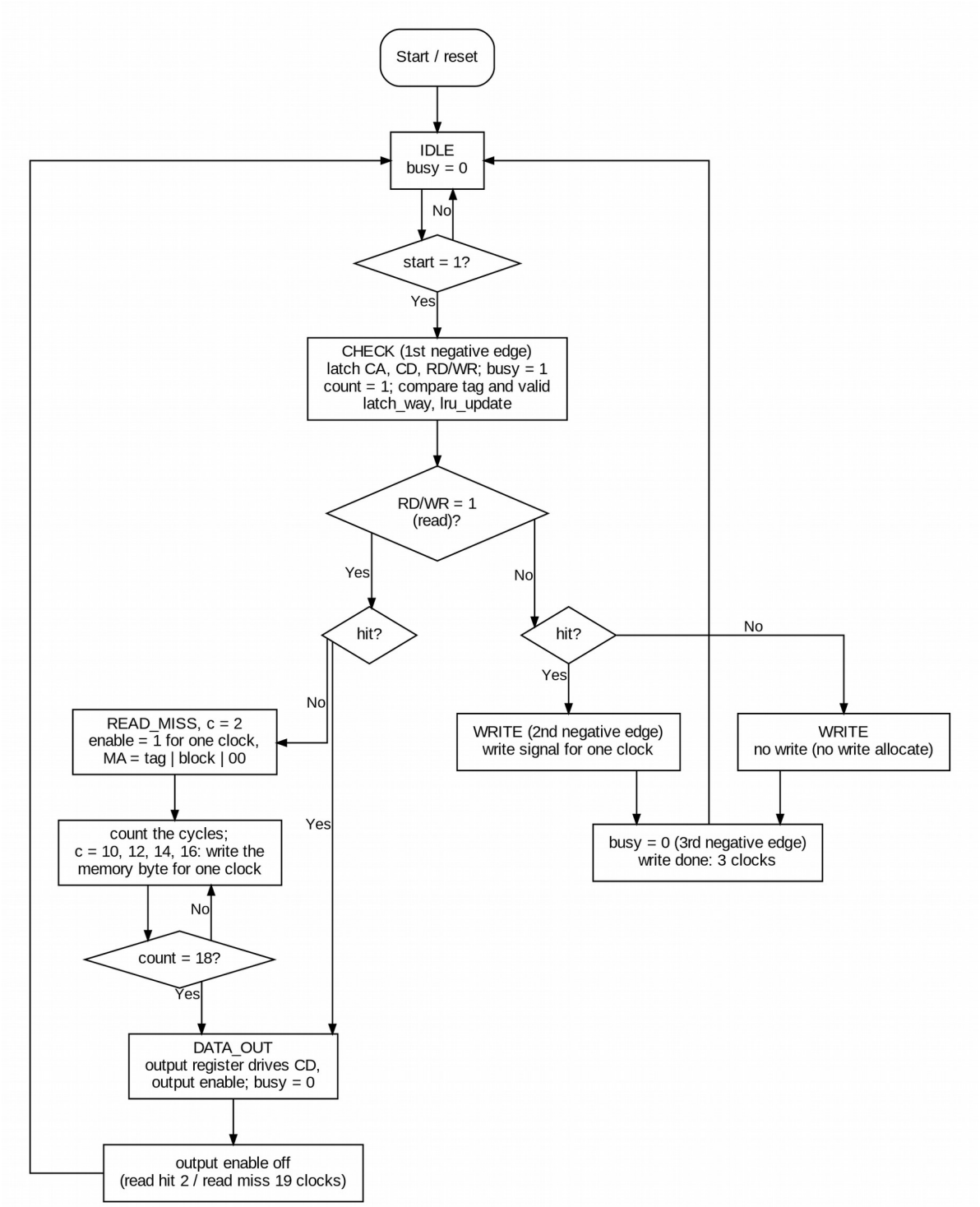


Figure 8: `state_machine`: operation flowchart

6.1 States

Table 9: States of the state machine

State	What happens	Why it is necessary
IDLE	Waits for start. busy = 0.	Reset state; the chip must not react until a request arrives.

CHECK	Entered on the negative edge after start: CA, CD and RD/WR are latched, busy goes high. During this cycle the addressed block's tag and valid bit and the addressed byte are read and compared (hit/miss) in both ways. On the negative edge that ends CHECK, latch_way keeps the victim way and lru_update marks the way used as most recently used.	The inputs are removed by the CPU on the next positive edge, so they must be latched; the hit/ miss decision needs one cycle.
WRITE	Write hit: the write signal writes CD into the addressed byte (positive edge to positive edge). Write miss: nothing is written (no write allocate). busy stays high.	The specification requires 3 clocks for a write hit or miss, with the write on the second negative edge.
READ_MISS	c = 2: enable = 1 with MA = tag, block offset, 00. c = 10, 12, 14, 16: the memory byte is valid, the write signal writes it (bytes 0–3; tag and valid with them). c = 18: the requested byte is read from the cache. The ways hold their tag, valid and read byte while the block is filled.	The memory needs 8 cycles and delivers one byte every 2 cycles; the block must be in the cache before the byte is returned.
DATA_OUT	The output register drives CD (output enable) for one clock; busy = 0. A new start can be accepted here.	The CPU reads the data on the positive edge after output enable; it must stay stable for one cycle.

Next-state equations (· = AND, + = OR):

- CHECK′ = start · (IDLE + DATA_OUT) · not reset
- WRITE′ = CHECK · not RD/WR · not reset
- READ_MISS′ = (CHECK · RD/WR · miss + READ_MISS · not c18) · not reset (c18 = count(4) · count(1))
- DATA_OUT′ = (CHECK · RD/WR · hit + READ_MISS · c18) · not reset
- IDLE′ = reset + (IDLE + DATA_OUT) · not start · not reset + WRITE
- latch_way = CHECK; lru_update = CHECK · (hit + RD/WR) (no LRU change on a write miss)
- count_enable = (CHECK + WRITE + READ_MISS) · not reset; count_restart = CHECK′ (the counter starts at 1 with every request)
- hit_result = the hit of the CHECK cycle (dff_en loaded in CHECK or reset), held until the next request's CHECK

6.2 Timing

Table 10: Timing of each operation

Negative edge after start	Read hit	Write hit / write miss	Read miss
1st	latch inputs, busy = 1 (CHECK)	latch inputs, busy = 1 (CHECK)	latch inputs, busy = 1 (CHECK)
2nd	data → output register, output enable, busy = 0	WRITE; write signal on next positive edge (hit only)	enable = 1, MA driven (READ_MISS, c = 2)
3rd	output enable off	busy = 0 (IDLE)	enable = 0, MA high impedance
10th, 12th, 14th, 16th			memory byte 0, 1, 2, 3 valid; written from the next positive edge for one cycle
18th			requested byte read from the cache (c = 18)
19th			data → output register, output enable, busy = 0
20th			output enable off

6.3 Inputs, outputs and connections

Table 11: State machine inputs and outputs

Signal	Dir	Connected to	Function
clk, reset	in	chip ports	Clock (negative edge) and synchronous reset
start	in	chip port	Request from the CPU
read_write	in	dff_en (RD/WR register)	Latched RD/WR of the request
hit	in	cache (its Hit/Miss logic)	Hit of the latched address (the cache holds it from the end of CHECK)
count	in	counter	Clock cycle of the request (1 in CHECK)
busy	out	chip port busy	Handshake to the CPU
enable	out	chip port mem_en; tx gates of MA	Memory read request; drives MA
output_enable	out	tx gates of CD	Drives the output register onto CD for one cycle
latch_input	out	register_n (CA, CD), dff_en (RD/WR)	Latch the CPU request
latch_output	out	register_n (output register)	Latch the byte read from the cache
count_enable	out	counter	Count in CHECK, WRITE and READ_MISS, clear otherwise
count_restart	out	counter	Start the count at 1

			when a request is accepted
fill	out	cache; mux2_n (write data)	Read miss: byte offset from the counter, data from MD, write tag and valid
fill_byte	out	cache (byte offset mux)	Read miss: byte of the block being written (from the counter)
hit_result	out	chip (hit, miss)	Hit/miss of the request, kept from the end of CHECK
write	out	cache	Write signal (one clock, positive edge to positive edge)
latch_way	out	cache (fill_way register)	Keep the victim way for a read miss
lru_update	out	cache (lru)	Mark the way used as most recently used
state	out	(observation)	One-hot state, for the waveforms

6.4 Testbench and simulation results

The testbench drives the state machine with the real counter and the same clock as the course testbench and checks busy, enable, output enable and the state after every negative edge, and the write signal after every positive edge (and in the middle of each write pulse), for all four scenarios.

Simulation result: **PASS** — `tb_state_machine`: all checks passed, simulation ended at 352 ns.

7 Chip (top level)

The chip connects the blocks: the CA, CD and RD/WR registers (loaded by latch_input), the state machine and counter, the multiplexer that chooses the cache's write data (CPU or memory), the two-way cache (which takes the whole CPU address, reports hit/miss and keeps the LRU state), the output register, and the transmission gates that drive CD (output enable) and MA (enable). MA is tag, block offset and two zeros taken from Gnd. Its ports are exactly those of the component in chip_test.vhd. The hit/miss of a request is decided once, at the end of CHECK: the cache's hit (tag_hit) and the state machine's hit_result both keep it until the next request.

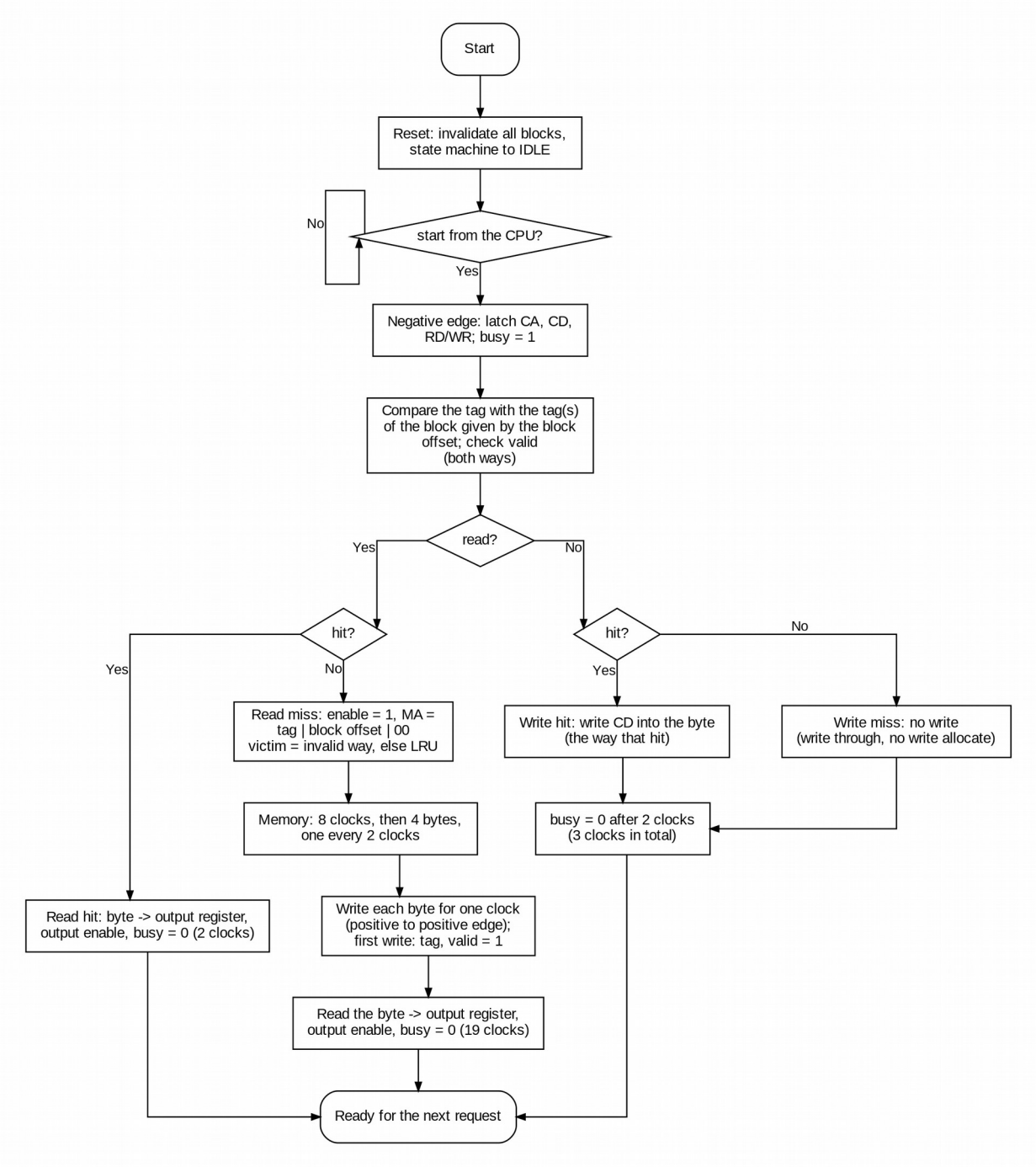


Figure 9: `chip`: operation flowchart

7.1 Extended chip testbench

- CPU model: drives start, RD/WR, CA (and CD) on a positive edge, removes them on the positive edge after busy, reads CD on the positive edge after busy goes low.
- Memory model: latches MA when enable is high on a positive edge and drives byte i of the block on the (8 + 2i)th negative edge after enable, stable for 2 cycles (byte at address A = 5A + 17).

- Reference model of the two-way cache (valid, tag and data of both ways, one LRU bit per block offset, victim = invalid way first, else LRU) predicts hit or miss and the data of every read; because the expected busy time depends on hit or miss, every hit/miss decision is checked.
- For every request it checks the data read, busy high on the 1st negative edge, busy low on the 2nd (read hit), 3rd (write) or 19th (read miss) negative edge, enable for exactly one cycle with the right MA on a read miss, MA and CD high impedance when not driven, and CD driven for exactly one cycle.
- Directed tests: the course sequence; two tags in block offset 0 kept in the two ways (0x00 and 0x10 both hit afterwards — in the direct mapped chip 0x10 would evict 0x00); a third tag (0x20) replacing the LRU way; the evicted block missing again; a write hit in way 1; write miss with no allocate; back-to-back requests; reset invalidating every block; then 150 random requests.

Simulation result: **PASS** — `tb_chip`: all checks passed, simulation ended at 15.22 µs.

Statistics: `requests=172 read hits=66 read misses=50 write hits=25 write misses=31`

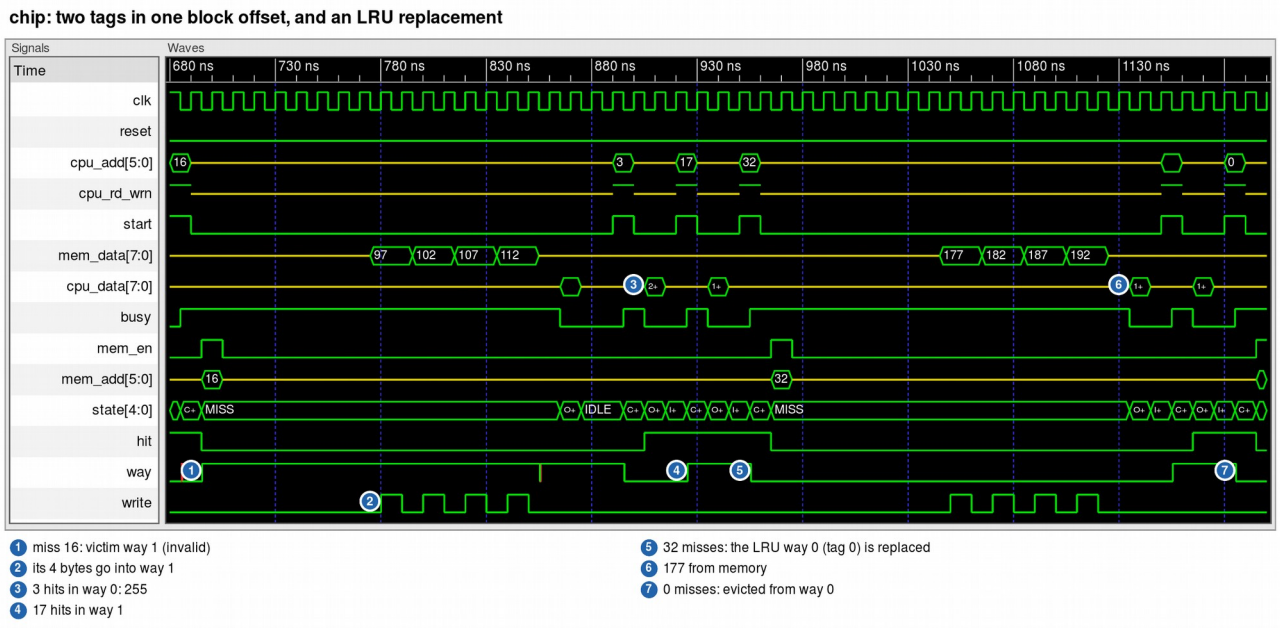


Figure 10: `tb_chip`: two tags in one block offset, and an LRU replacement

Read 0x10 (tag 01, block offset 0) misses while tag 00 is in way 0: way 1 is invalid, so it is the victim (`way` = 1 during the fill). Read 0x03 then hits in way 0 (FF, written earlier) and 0x11 hits in way 1 — both blocks of block offset 0 stay in the cache. Read

0x20 (tag 10) misses; both ways are valid and way 0 is the least recently used, so way 0 is filled.

8 Course testbench (chip_test)

The course testbench `chip_test.vhd` was run without any change: its chip component already has the same port names, types and order as this design's `chip` entity, and it binds `work.chip(structural)`. It reads `chip_in.txt` from the run directory and prints the chip's inputs and outputs on every positive and negative clock edge. It was run for 360 ns, as in `run.ncsim`.

8.1 Sample input

`chip_in.txt` holds six lines per clock: CA, CD, RD/WR, start and reset (applied on the positive edge) and MD (applied on the negative edge). One row per clock:

Table 12: chip_in.txt (inputs of the course testbench)

Clock	CA	CD	RD/WR	Start	Reset	MD
1	ZZZZZZ	ZZZZZZZZ	Z	0	1	ZZZZZZZZ
2	ZZZZZZ	ZZZZZZZZ	Z	0	1	ZZZZZZZZ
3	000000	ZZZZZZZZ	1	1	0	ZZZZZZZZ
4	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
5	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
6	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
7	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
8	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
9	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
10	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
11	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
12	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000000
13	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000000
14	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000001
15	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000001
16	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000010
17	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000010
18	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000011
19	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000011
20	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
21	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
22	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
23	000011	11111111	0	1	0	ZZZZZZZZ
24	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
25	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
26	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
27	000011	ZZZZZZZZ	1	1	0	ZZZZZZZZ
28	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
29	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
30	111111	10101010	0	1	0	ZZZZZZZZ
31	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
32	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
33	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
34	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
35	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ

Requests: reset for 2 clocks; read 0x00 (read miss; memory returns 00, 01, 02, 03); write 0xFF to 0x03 (write hit); read 0x03 (read hit, returns FF); write 0xAA to 0x3F (write miss).

8.2 Output and comparison with the reference

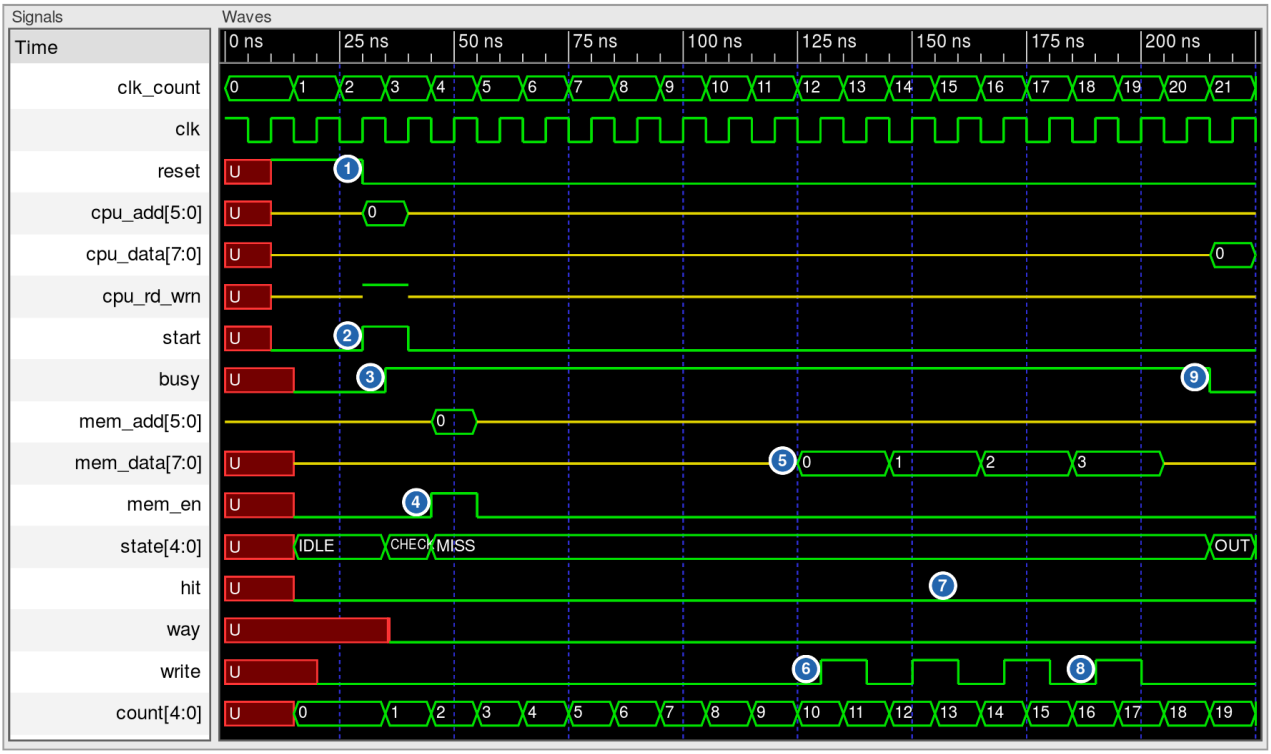
The output of this design (two_way/testbench/chip_out.log) was compared field by field with the course's reference ncsim.log. The reference was produced with 8-bit addresses (as its README notes), so only the low 6 address bits are compared. Result: MATCH in all 70 prints (differences only before the first reset edge: X in the reference, U in GHDL). The course sequence never puts two tags in one block offset, so the two-way chip produces exactly the same output as the direct mapped chip, with the same timing.

8.3 Waveforms

The course run is shown one operation per figure (read miss, write hit, read hit, write miss), followed by the clock cycle counter over the four requests. Each figure is drawn from the simulation of chip_test.vhd with the current design, with numbered markers on the events, explained with their clock number (clk_count) in the key under each figure.

All waveform figures in this report are drawn from the GHDL simulation results (VCD files), in the layout of a waveform viewer: a Signals panel with the names and a black Waves panel with a time ruler. Green is a valid 0/1 or bus value (buses in decimal), a red box is U or X, and a yellow line in the middle is Z (bus released); a value too long for its box is cut with a +. The one-hot state is shown with its state name. The chip_test figures use the signal names of chip_test.vhd: clk_count = CLOCK_COUNT, cpu_add = CPU_ADDRESS (CA), cpu_data = CPU_DATA (CD), cpu_rd_wrn = READ_WRITE, mem_add = MEMORY_ADDRESS (MA), mem_data = MEMORY_DATA (MD), mem_en = MEMORY_ENABLE; state, hit, way, write and count are inside the chip.

Read miss: reset, then a read of address 0 (19 clocks)



1. clock 2: reset released: every valid bit is 0, state IDLE
2. clock 2: start with CA = 0, RD/WR = 1 (read)
3. clock 3: busy rises; CHECK compares the tag: miss
4. clock 4: READ_MISS: enable for one clock, MA = 0
5. clock 12: memory byte 0 (value 0), 8th negative edge after enable
6. a write pulse stores it; bytes 1-3 follow every 2 clocks
7. hit stays 0 for the whole read miss (held from CHECK)
8. clock 18: byte 3 written: the block is valid with its tag
9. clock 21: busy falls, 0 on CD; count = 19: 19 clocks

Figure 11: chip_test: read miss (read of 0x00 after reset, 19 clocks)

Reset for two clocks; the read of 0x00 starts on clock 2. busy goes high on the next negative edge (clock 3); on the miss, MEMORY_ENABLE is on for one clock with MEMORY_ADDRESS = 0 (clock 4). The memory returns bytes 0, 1, 2, 3 from clock 12, each for 2 clocks; each is written into block 0 by a one-cycle write pulse (positive edge to positive edge) while it is stable. On clock 21 busy goes low and the data 0 is on CPU_DATA for one clock: 19 clocks (count = 19). hit, the request's hit/miss, stays 0 from the CHECK cycle to the end of the request: the tag and valid cells being written during the fill do not reach it. The four operation figures are the GHDL simulation of the course testbench chip_test.vhd with the course input file. Compared with the course's desired waveforms (cmpe413_project_desired_waveforms.pdf), every event falls on the same clock: busy high on clock 3, memory enable and MA = 00 on clock 4, the memory bytes 00, 01, 02, 03 from clock 12 every 2 clocks, busy low with 00 on CPU data on clock 21 (19 clocks); write hit 3 clocks, read hit 2 clocks with FF, write miss 3 clocks with no memory access (addresses are 6 bits in this design).

Write hit: write 255 to address 3 (3 clocks)

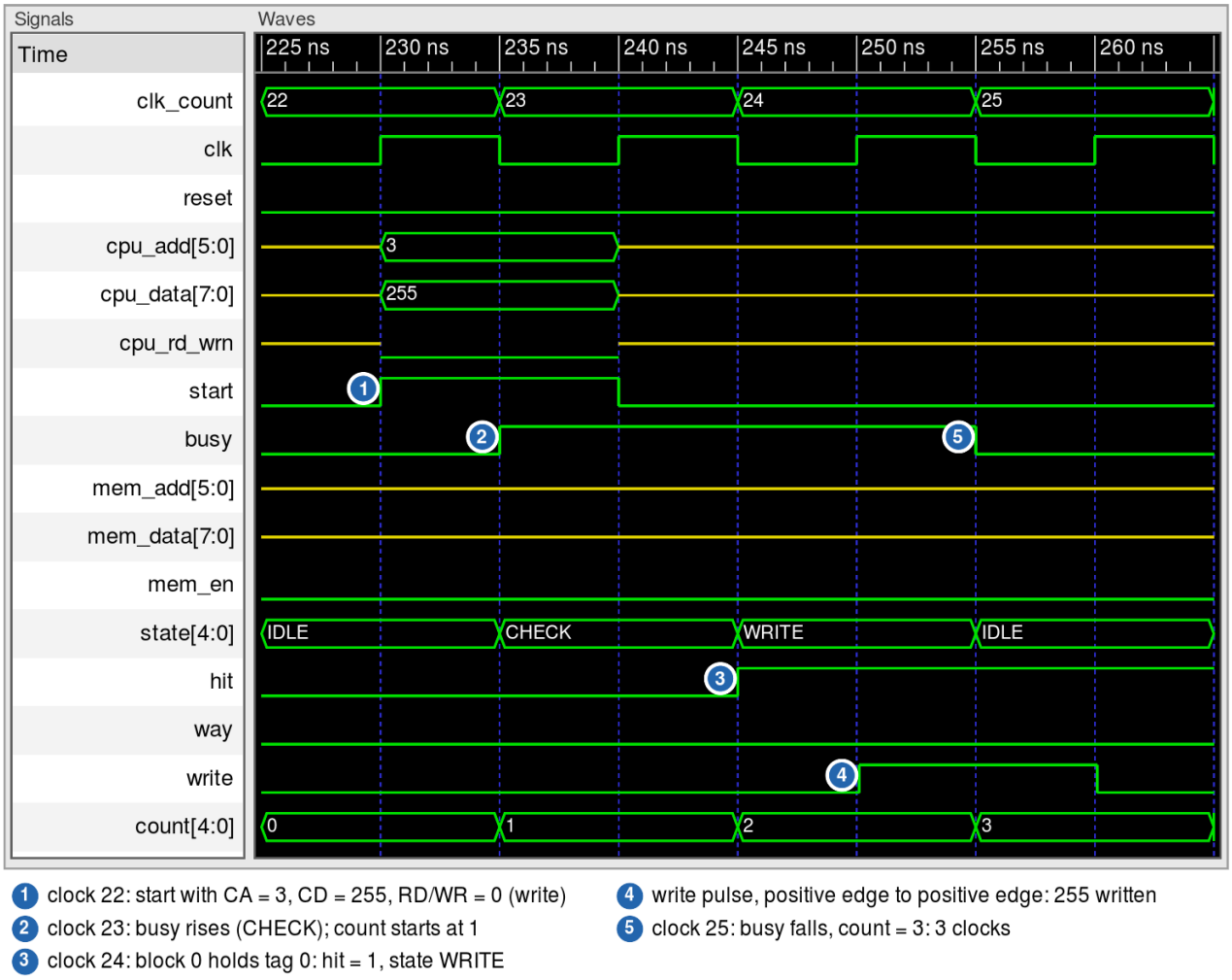


Figure 12: chip_test: write hit (0xFF to 0x03, 3 clocks)

Write 0xFF (255) to 0x03: the block is already in the cache, so CHECK finds hit = 1 and the state machine goes to WRITE; the write signal is on for one clock, from the positive edge after the 2nd negative edge to the next positive edge, and busy goes low on the 3rd negative edge: 3 clocks (count = 3). The counter started again at 1 with this request.

Read hit: read address 3 (2 clocks)

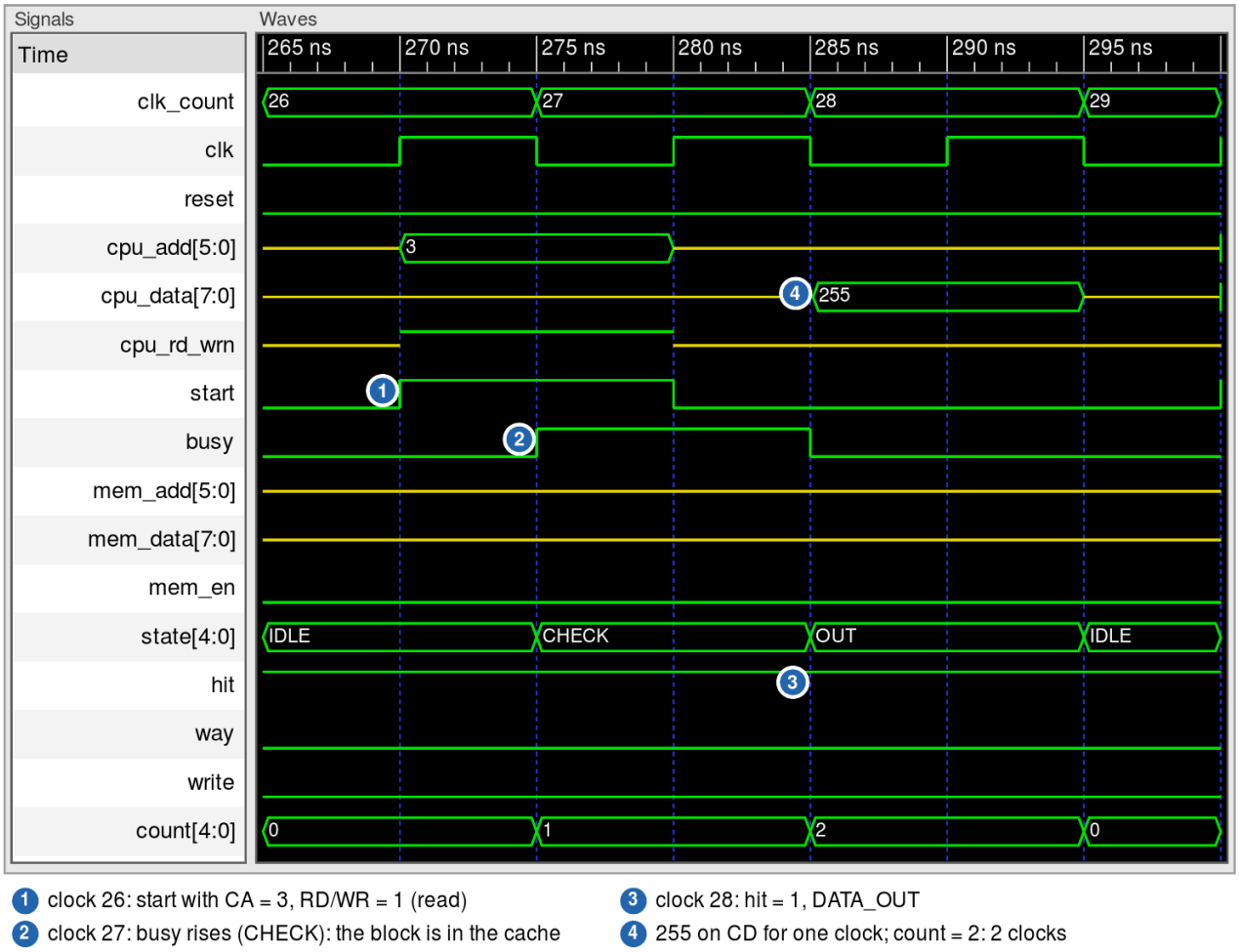


Figure 13: chip_test: read hit (0x03, 2 clocks)

Read 0x03: busy for one clock (CHECK, hit = 1), then DATA_OUT puts 255 (0xFF, written by the write hit) on CPU_DATA for one clock while busy is low: 2 clocks (count = 2).

Write miss: write 170 to address 63 (3 clocks)

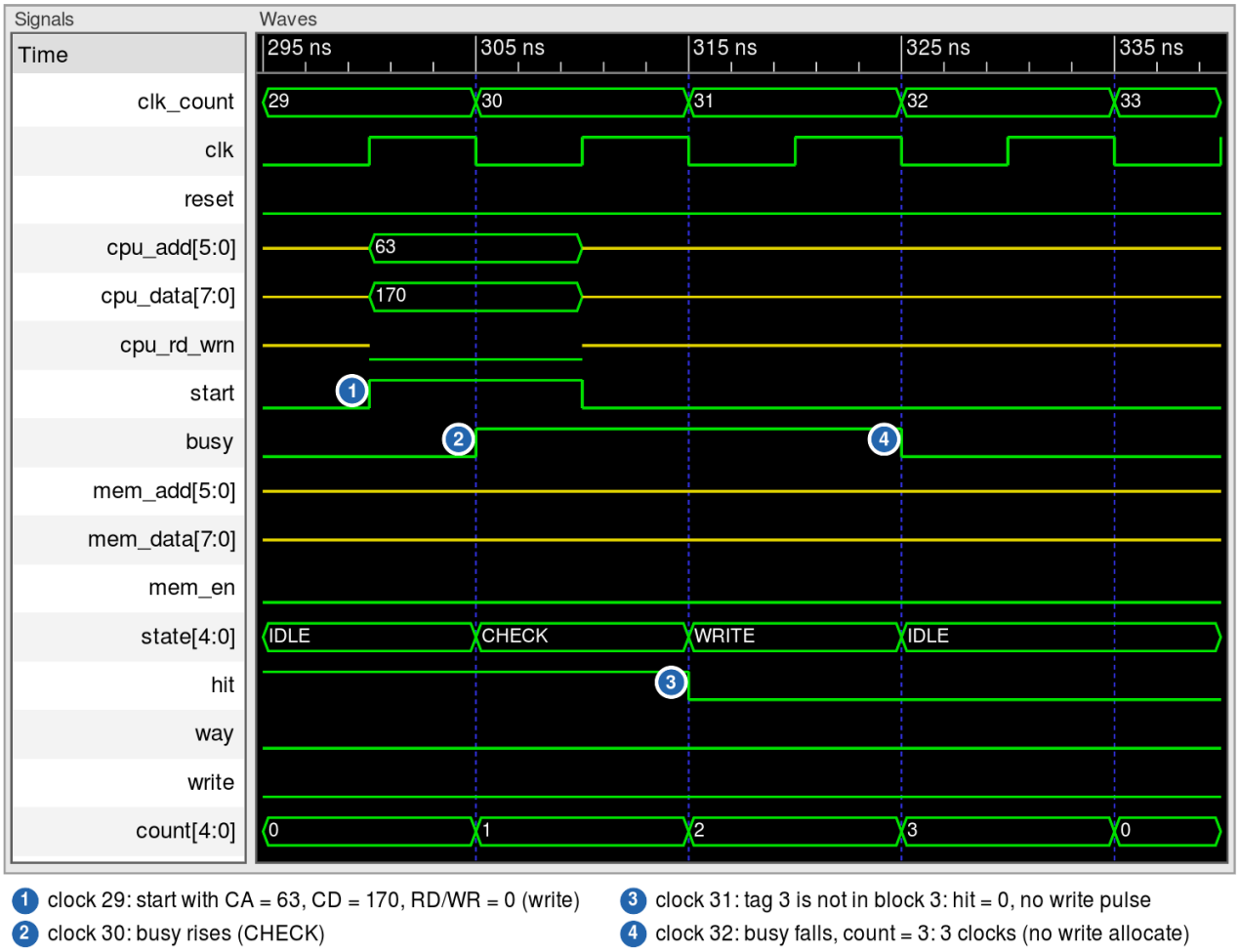


Figure 14: chip_test: write miss (0xAA to 0x3F, 3 clocks)

Write 0xAA (170) to 0x3F (63): tag 11 is not in block 3 of either way, so hit = 0 and no write pulse is made (write through with no write allocate: the cache is not changed); busy for 2 clocks, low on the 3rd negative edge: 3 clocks (count = 3).

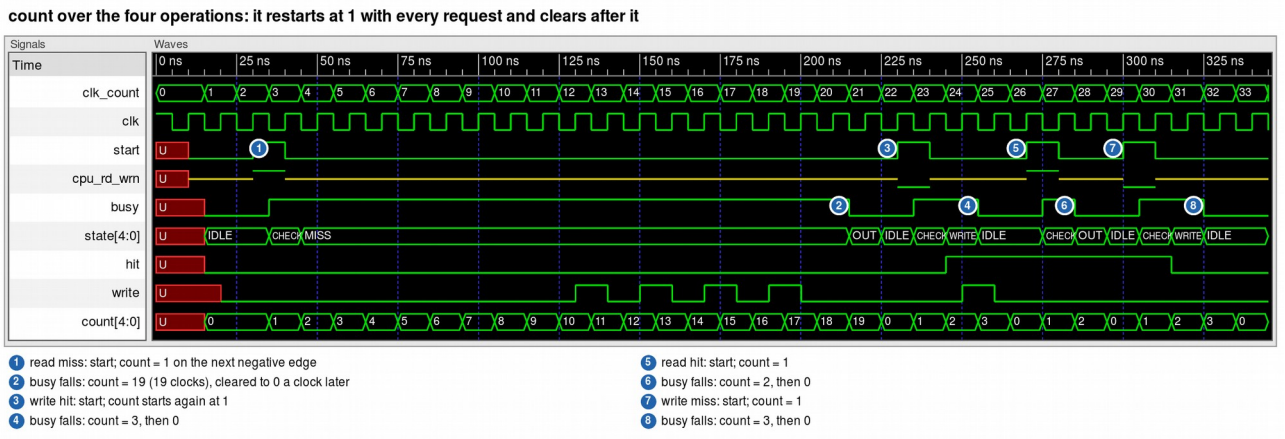


Figure 15: chip_test: the clock cycle counter over the four operations

The same run as the four figures above (chip_test with the course input file). count is started at 1 on the negative edge that accepts a request (busy goes high), counts while busy is high, and after the edge where busy goes low holds the request's length: 19 (read miss), 3 (write hit), 2 (read hit), 3 (write miss). One clock later it is cleared to 0; a request that follows at once starts it at 1 again.

9 Notes for the TA: running the simulations

The design is in the folder code/two_way/ of the submission: the VHDL in primitives/, gates/, blocks/, cache/, control/, cache_pkg.vhd and chip.vhd; the testbenches, the course testbench chip_test.vhd, its input file chip_in.txt and its output chip_out.log in testbench/. The code, this report and a README with these instructions are also on GitHub: https://github.com/aali12-eng/CMPE413-Cache-Chip-2Way . The simulations run in Cadence Xcelium with SimVision, as in the course lab: load the Cadence environment so that xrun is on the PATH, then from code/two_way run the script, or the same steps by hand from code/two_way/testbench. The design is compiled as VHDL-93; chip_test.vhd uses VHDL-87 file declarations, so it is compiled without -v93.

Testbench	Simulates	Input	Output
chip_test (course)	chip	two_way/testbench/ chip_in.txt (format: Section 8.1)	printed on every clock edge → chip_out.log
tb_chip	chip	built in (CPU, memory and reference models)	PASS/FAIL report + statistics
tb_state_machine	state_machine + counter	built in	PASS/FAIL report
tb_cache, tb_cache_block, tb_hit_miss	cache blocks	built in	PASS/FAIL report
tb_selector, tb_cache_cell,	building blocks	built in	PASS/FAIL report

tb_register_n, tb_counter			

9.1 Assumptions

- Write through: the chip interface has no memory write port (MA is driven only with enable, MD is an input), so writes update the cache on a hit and the memory side of write through is outside this chip.
- Gate delay 100 ps (Section 3.2); the course primitives have zero delay, as provided.
- The CPU follows the protocol of the specification: it removes its signals on the positive edge after busy, and starts a new request no earlier than the positive edge after busy goes low.

10 Breakup of work

Table 13: Breakup of work

Task	Asif Ali (RE16458)
Cache cell, cache block, cache	Done
State machine and counter	Done
Chip top level	Done
Testbenches and simulation	Done
Report	Done